CYBERSECURITY PROJECT REPORT

Net Guardian: Network Intrusion Detection System

Rule-Based Packet Monitoring with Snort, AI-Assisted Rule Authoring & Live Interactive Console

Project Title: Net Guardian — Network Intrusion Detection Console	**Submitted By : N. Sri Chaithanya**
Live Published Dashboard: https://netguardian.freebuff.app/	**Core Detection Engine:** Snort (Network-Based Intrusion Detection System)
Dashboard & Rule Support: Freebuff App Builder & AI Agent Rule Assistance	**Submission Date:** November 2026

01 Introduction: What is a Network Intrusion Detection System?

A Network Intrusion Detection System (NIDS) is a dedicated cybersecurity monitoring solution that inspects live network packets in real time to detect malicious activity, unauthorized access attempts, and policy violations. Acting like a security camera for network traffic, a NIDS analyses packet headers and payloads across network segments and raises alerts the moment suspicious patterns appear.

Unlike traditional firewalls that strictly block or allow traffic based on static port and IP rules, a NIDS performs deep packet inspection against known attack signatures and behavioural anomalies. In this project, I designed and published Net Guardian (accessible at https://netguardian.freebuff.app/), a complete intrusion detection workflow powered by Snort on the backend and an interactive visual monitoring console on the frontend.

02 Project Objectives: Why This System Was Built

Modern networks face continuous automated reconnaissance, port scans, brute-force logins, and malware command-and-control traffic. Raw terminal logs generated by command-line security tools can be overwhelming to read during an active security event. This project was built to fulfill five core goals:

- **Deploy a Network-Based IDS Engine:** Configure Snort to capture and inspect live network traffic across monitored interfaces.
- **Author Custom Detection Rules & Alerts:** Write precise signature rules to identify reconnaissance scans, ICMP floods, suspicious SSH/HTTP probes, and unauthorized access attempts.
- **Continuous Traffic Monitoring:** Maintain non-stop packet inspection to spot emerging threats before they escalate into full system compromises.
- **Implement Incident Response Mechanisms:** Establish structured alert severity levels, IP flagging, and containment workflows whenever an intrusion rule triggers.
- **Visualize Security Events Clearly:** Present detected alerts, traffic trends, and attack categories through a clean, published web dashboard for rapid decision-making.

03 System Architecture & Implementation Using Snort

The core detection architecture separates heavy packet inspection from visual reporting so each layer performs efficiently:

3.1 Backend Packet Inspection with Snort

Snort was deployed as the primary backend intrusion detection engine. Configured in Network Intrusion Detection (NIDS) mode, Snort listens continuously to incoming and outgoing packets on the network interface, decodes Ethernet/IP/TCP/UDP/ICMP headers, normalizes application payloads, and matches traffic against our local ruleset (local.rules). Whenever a packet matches a signature, Snort logs the event with timestamps, source/destination IP addresses, port numbers, protocol metadata, and priority classifications.

3.2 Rule Authoring with AI Agent Assistance

Writing accurate Snort rule syntax requires careful handling of protocol headers, content flags, flow states, and threshold limits to prevent false positives. To build reliable detection rules efficiently, I collaborated with an AI agent to structure, validate, and refine custom Snort signatures before deploying them into the Snort backend. Below are representative examples of the rules implemented in the detection engine:

```
# 1. Detect Nmap SYN Stealth Port Scan Activity
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"NIDS ALERT: Potential TCP SYN Port Scan"; flags:S;
threshold:type threshold, track by_src, count 15, seconds 5; sid:1000001; rev:1;)


# 2. Detect ICMP Ping Flood / Reconnaissance Sweep
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"NIDS ALERT: High-Rate ICMP Ping Flood Detected";
itype:8; threshold:type both, track by_src, count 20, seconds 3; sid:1000002; rev:1;)


# 3. Detect Repeated SSH Brute-Force Connection Attempts
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"NIDS ALERT: Possible SSH Brute Force Attempt";
flags:S; threshold:type threshold, track by_src, count 6, seconds 30; sid:1000003; rev:1;)


# 4. Detect Suspicious SQL Injection Pattern in HTTP URI alert tcp $EXTERNAL_NET any -> $HOME_NET 80
(msg:"NIDS ALERT: SQL Injection Signature in HTTP Request"; flow:to_server,established; content:"UNION
SELECT"; nocase; http_uri; sid:1000004; rev:1;)
```

3.3 Continuous Monitoring & Response Mechanisms

- **Continuous Real-Time Capture:** Snort runs persistently in daemon/console mode, ensuring every packet traversing the monitored subnet is inspected without manual intervention.
- **Severity-Based Alert Triage:** Alerts are categorized by priority (Critical, High, Medium, Low) based on threat impact so urgent attacks stand out immediately.
- **Automated & Manual Response Workflow:** When an intrusion is triggered, the system logs the offending source IP, flags the session for firewall rule blocking, and surfaces actionable remediation steps on the console.

04 Frontend Visualization & Honest Rationale for Tool Selection

---

To make the intrusion detection logs practical for real-world monitoring, the backend alert output is visualized through a live web console published at https://netguardian.freebuff.app/.

> **Transparent Note on Why AI Assistance & Freebuff Were Used**
>
> I am primarily a cybersecurity learner rather than a full-stack web developer, so writing complex frontend web code (HTML, CSS, JavaScript, and charting frameworks) from scratch was outside my core programming skillset.
>
> Instead of leaving the project as raw terminal text logs, I took a practical, problem-solving approach: I configured Snort as the core backend engine, used an AI agent to assist in writing and validating accurate Snort detection rules, and built and published the interactive frontend dashboard using Freebuff

(https://netguardian.freebuff.app/). This allowed me to focus deeply on network security concepts, rule logic, and threat analysis while still delivering a polished, accessible monitoring interface.

4.1 Summary of Tools & Technologies Used

Layer / Role	Tool / Platform Used	Purpose in Net Guardian
Backend NIDS Engine	**Snort**	Captures live network packets, performs deep packet inspection, and triggers alerts on malicious signatures.
Rule Authoring Support	**AI Agent**	Assisted in drafting, debugging, and refining custom Snort detection rules for scans, floods, and exploits.
Frontend Dashboard	**Freebuff**	Used to design, host, and publish the live Net Guardian web console without manual frontend coding.
Live Deployment	**netguardian.freebuff.app**	Provides browser-based visibility into detected threats, severity metrics, and response actions.

05 Overall Summary, Key Learnings & Conclusion

5.1 What I Learned Through This Project

- **Core Understanding of Intrusion Detection:** I gained hands-on clarity on what a Network Intrusion Detection System actually does—how network packets carry attack fingerprints and how signature rules distinguish normal user traffic from malicious reconnaissance or exploitation.
- **How Snort Rules Work Under the Hood:** By crafting and testing custom rules with AI guidance, I learned how rule headers (action, protocol, source/destination IPs and ports) and rule options (message, content, flags, threshold, Sid) work together to catch specific threats accurately.
- **Turning Raw Logs into Actionable Visibility:** I learned that security detection is only effective when alerts are readable and actionable. Integrating Snort's backend detection logic with a visual dashboard on Freebuff demonstrated how security operations teams monitor threats in real time.
- **Practical Problem-Solving with Modern Tools:** Even without a traditional software development background, combining foundational security tools (Snort) with modern AI assistance and no-code/low-code platforms (Freebuff) enabled me to build and deploy a functional end-to-end security solution.

5.2 Overall Summary & Conclusion

The Net Guardian project successfully fulfills every objective of a modern Network Intrusion Detection System. By pairing Snort's packet-level inspection engine with AI-assisted custom detection rules and

publishing a live visual console on Freebuff (https://netguardian.freebuff.app/), the system provides continuous traffic monitoring, clear threat alerting, and structured incident response guidance.

Overall, building and publishing Net Guardian bridged the gap between theoretical network security concepts and practical defensive operations, resulting in a working, user-friendly intrusion detection console ready for real-world demonstration.